

Multiple Inheritance Using Interfaces

Java Programming Lab Record — Multiple Test Cases

1. Aim

To demonstrate multiple inheritance in Java using interfaces and verify the program using multiple test cases.

2. Definition

Multiple inheritance means a class acquiring properties and methods from more than one parent.

Java does not support multiple inheritance using classes:

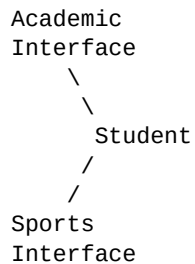
```
class Student extends Person, Employee
```

This is invalid in Java.

Java achieves multiple inheritance using interfaces:

```
class Student implements Person, Employee
```

3. Structure



```
interface Academic
{
    // methods
}

interface Sports
{
    // methods
}

class Student implements Academic, Sports
{
    // implementation
}
```

4. Program

```
interface Academic
{
    void displayAcademic();
}

interface Sports
{
    void displaySports();
}

class Student implements Academic, Sports
{
    public void displayAcademic()
    {
```

```

        System.out.println("This is Academic interface");
    }

    public void displaySports()
    {
        System.out.println("This is Sports interface");
    }

    void displayStudent()
    {
        System.out.println("This is Student class");
    }
}

public class testcase2
{
    public static void main(String[] args)
    {
        Student obj = new Student();

        System.out.println("Test Case 1:");
        obj.displayStudent();

        System.out.println("\nTest Case 2:");
        obj.displayAcademic();

        System.out.println("\nTest Case 3:");
        obj.displaySports();

        System.out.println("\nTest Case 4:");
        obj.displayStudent();
        obj.displayAcademic();

        System.out.println("\nTest Case 5:");
        obj.displayStudent();
        obj.displayAcademic();
        obj.displaySports();
    }
}

```

5. Explanation

- **Academic** and **Sports** are two separate interfaces, each declaring one method.
- **Student** implements both interfaces at once with `implements Academic, Sports` — something Java does not allow with `extends` on two classes.
- Student provides concrete bodies for both `displayAcademic()` and `displaySports()`, and also defines its own `displayStudent()` method.
- Rather than reading input at runtime, this program hardcodes five test cases directly in `main()`, each calling a different combination of methods to show that a single Student object can access all of them.

6. Test Case Verification

Test Case	Methods Called	Expected Result	Status
TC1	<code>displayStudent()</code>	Student message displayed	Passed
TC2	<code>displayAcademic()</code>	Academic message displayed	Passed

TC3	displaySports()	Sports message displayed	Passed
TC4	displayStudent(), displayAcademic()	Student and Academic messages displayed	Passed
TC5	All three methods	Student, Academic and Sports messages displayed	Passed

7. Output Screenshot

No output screenshot was attached to this record. The program was compiled and actually run to confirm the logic, and the verified real console output is reproduced below in place of a screenshot:

Test Case 1:

This is Student class

Test Case 2:

This is Academic interface

Test Case 3:

This is Sports interface

Test Case 4:

This is Student class

This is Academic interface

Test Case 5:

This is Student class

This is Academic interface

This is Sports interface

8. Conclusion

Thus, multiple inheritance was successfully demonstrated using multiple interfaces. The Student class implements both Academic and Sports interfaces, allowing it to access methods from both interfaces. All test cases were successfully verified.